

Exercise 1: (6 pts)

A "counter" has a name (TicTic for example) and it counts from 5 to n (any positive integer number). It marks a random pause after each number except the last one.

A counter displays each number (TicTic will display for example, "TicTic > 9") and it displays a message of type "/!\ TicTic has finished counting till 100" when it has finished.

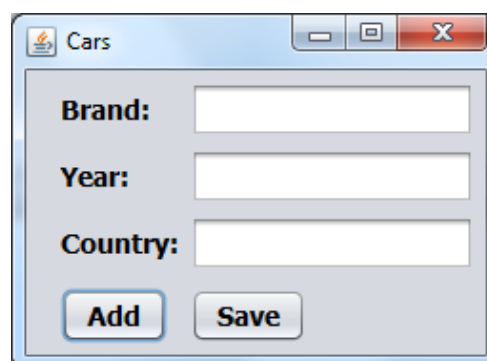
Write the class « Counter » and test it by running many counters that count till 100. Show the counter that has finished first.

Exercise 2: (14 pts)

Consider the following class named *Car*:

```
public class Car {  
    private String brand;  
    private int year;  
    private String country;  
  
    public Car(String brand, int year, String country) {  
        this.brand = brand;  
        this.year = year;  
        this.country = country;  
    }  
  
    public int getYear() {  
        return year;  
    }  
  
    public String getCountry() {  
        return country;  
    }  
  
    public String getBrand() {  
        return brand;  
    }  
}
```

Consider the following graphical interface that allows the data input of a *Car* using text fields. The button *Add* allows the creation of a new *Car* object and adds it to a *Vector*. The button *Save* allows the saving of the objects *Car* of the *Vector* to a file. The file's data are loaded at program start-up using the method *load* below.

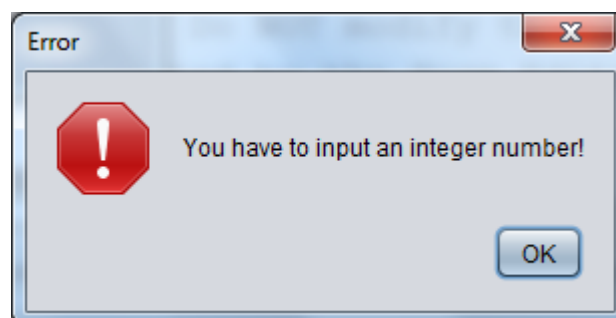


The corresponding code of this graphical interface is as follows:

```
public class CarFrame extends Frame {  
    private TextField brand = new TextField(10);  
    private TextField year = new TextField(10);  
    private TextField country = new TextField(10);  
    private Label lBrand = new Label("Brand:");  
    private Label lYear = new Label("Year:");  
    private Label lCountry = new Label("Country:");  
    private Button add = new Button("Add");  
    private Button save = new Button("Save");  
    private Vector carsList = new Vector();  
}
```

```
public CarFrame() {  
    // to complete ... (1)  
    this.pack();  
    load();  
    // to complete ... (2)  
}  
  
private void addPerformed() {  
    // to complete ... (3)  
}  
  
private void savePerformed() {  
    // to complete ... (4)  
}  
  
public void save2Performed() {  
    // to complete ... (5)  
}  
  
private void load() {  
    // to complete ... (6)  
}  
}
```

- a) Give in (1) the necessary code in order to obtain the above graphical interface. (You may use the layout managers *BorderLayout*, *FlowLayout*, and *GridLayout*).
- b) Give in (2) the code that adds an *ActionListener* to the button *Add* and executes the method *addPerformed*.
- c) Give in (3) the code that allows the data input in the text fields to create an object *Car*, and to add it to the object *carsList*. The text fields should be empty after the addition.
- d) Give in (4) the code that opens the file « car.txt » and to write the content of the *carsList* to it. Each line must contain the data of one *Car* object as follows:
Mercedes,1990,Germany
- e) Give in (5) the code that serializes (persists) the *Car* objects of the *carsList* to a file « data.bin ». Which change should you do to the class *Car* regarding the persistence?
- f) Give in (6) the code that loads the content of the file « car.txt », creates the *Car* objects (one by line), and adds them to the *carsList*.
- g) The method *parseInt* raises an exception of type *NumberFormatException* if the String passed as parameter cannot be converted to an integer number. Change the code of the method *addPerformed()* in order to support this exception. The raise of the exception will display a *JOptionPane* as follows:



Exercice 1: (6 pts)

Un "compteur" a un nom (TicTic par exemple) et il compte de 5 à n (nombre entier positif quelconque). Il marque une pause aléatoire après chaque nombre sauf le dernier.

Un compteur affiche chaque nombre (TicTic affichera par exemple, "TicTic > 9") et il affiche un message du type "/!\ TicTic a fini de compter jusqu'à 100" quand il a fini.

Écrire la classe « Compteur » et tester-la en lançant plusieurs compteurs qui comptent jusqu'à 100. Montrer celui qui a fini le premier.

Exercice 2: (14 pts)

On considère la classe *Car* suivante:

```
public class Car {                // voiture en français
    private String brand;          // marque en français
    private int year;              // année en français
    private String country;        // pays en français

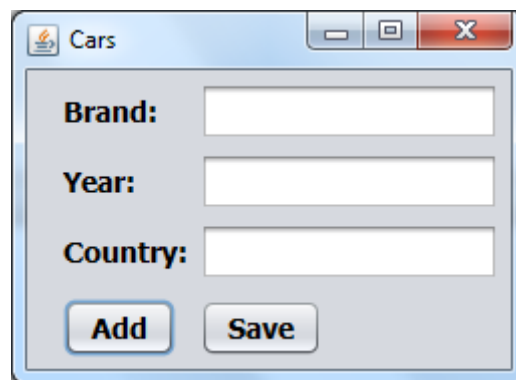
    public Car(String brand, int year, String country) {
        this.brand = brand;
        this.year = year;
        this.country = country;
    }

    public int getYear() {
        return year;
    }

    public String getCountry() {
        return country;
    }

    public String getBrand() {
        return brand;
    }
}
```

On considère l'interface graphique suivante permettant de saisir les données d'une voiture dans des champs de texte. Le bouton *Add* permet de créer un nouvel objet *Car* et de l'ajouter à un *Vector*. Le bouton *Save* permet de sauvegarder les voitures du *Vector* dans un fichier. Les données du fichier sont chargées au démarrage du programme à l'aide de la méthode *load* ci-dessous.



Le code correspondant à cette interface graphique est comme suit:

```
public class CarFrame extends Frame {
    private TextField brand = new TextField(10);
    private TextField year = new TextField(10);
    private TextField country = new TextField(10);
    private Label lBrand = new Label("Brand");
    private Label lYear = new Label("Year");
    private Label lCountry = new Label("Country");
    private Button add = new Button("Add");
    private Button save = new Button("Save");
    private Vector carsList = new Vector();
}
```

```
public CarFrame() {  
    // à compléter ... (1)  
    this.pack();  
    load();  
    // à compléter ... (2)  
}  
  
private void addPerformed() {  
    // à compléter ... (3)  
}  
  
private void savePerformed() {  
    // à compléter ... (4)  
}  
  
public void save2Performed() {  
    // à compléter ... (5)  
}  
  
private void load() {  
    // à compléter ... (6)  
}  
}
```

- a) Donner en (1) le code nécessaire afin d'obtenir l'interface graphique ci-dessus. (Vous pouvez utiliser les gestionnaires *BorderLayout*, *FlowLayout*, et *GridLayout*).
- b) Donner en (2) le code qui permet d'ajouter un écouteur d'action (*ActionListener*) au bouton *Add* et qui exécute la méthode *addPerformed*.
- c) Donner en (3) le code permettant de saisir les champs de texte, de créer un objet *Car*, et de l'ajouter à l'objet *carsList*. Les champs doivent être vides après l'ajout.
- d) Donner en (4) le code permettant d'ouvrir le fichier « car.txt » et d'y écrire les informations des livres de la *carsList*. Chaque ligne doit contenir les informations d'une seule voiture comme suit :
Mercedes, 1990, Germany
- e) Donner en (5) le code permettant de sérialiser (persister) les objets *Car* de la *carsList* dans un fichier « data.bin ». Quel changement doit-on effectuer à la classe *Car* pour la persistance?
- f) Donner en (6) le code permettant de charger les informations contenues dans le fichier « car.txt », de créer des objets *Car* (un par ligne) et de les ajouter à la *carsList*.
- g) La méthode *parseInt* soulève une exception du type *NumberFormatException* si la String passée en paramètre ne peut pas être convertie en un nombre entier. Changer le code de la méthode *addPerformed()* afin de supporter cette exception. L'appel de cette exception affichera un *JOptionPane* comme suit:

